Brain Tumour Detection

Using Deep Learning & VGG16 Transfer Learning

A Complete Beginner-Friendly Study Guide

What You'll Learn
Understand every step — from loading MRI images to predicting tumor types using a powerful AI model.

Project Source
Based on YouTube tutorial by AI with Noor | youtu.be/MTQQnziWGQY

Target Audience
Students & beginners who want to demo & explain this project confidently.

■	**6 Chapters**	Complete project walkthrough
■	**Architecture Diagram**	Visual model explanation
■	**Web Integration**	How to connect model to website
■	**20 Q&A;**	Interview/demo prep questions
■	**GitHub Code**	Clean, human-style .ipynb file
■	**Demo Script**	How to present on webcam

Table of Contents

Chapter 1

Project Overview

What Is This Project About?

This project builds an AI system that looks at a brain MRI (scan) image and tells us whether the person has a brain tumour — and if yes, which type. It uses a technique called **Deep Learning**, which is a way of teaching computers to recognise patterns in images, similar to how a doctor learns to read scans after years of training.

1.1 Brain Tumours — A Simple Explanation

A brain tumour is an abnormal growth of cells inside the brain. Not all tumours are equally dangerous. This project classifies four types:

Type	Plain English Meaning	Danger Level
Glioma	Tumour from glial (support) cells — most common	High
Meningioma	Tumour from the brain's protective lining	Medium
Pituitary	Tumour in the pituitary gland at brain's base	Variable
No Tumor	Completely healthy MRI scan	None

1.2 Why Deep Learning?

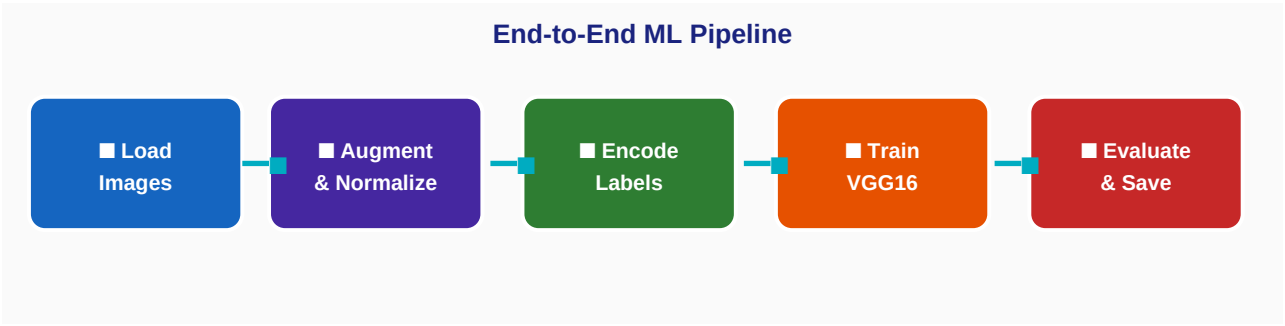
■ Advantages

- Automatically learns features — no manual rules
- Can process thousands of images quickly
- Matches (sometimes beats) radiologist accuracy
- Scales easily to new data

■■ Challenges

- Needs lots of data to train well
- Acts like a black box (hard to explain)
- Requires GPU for fast training
- Must retrain if new tumour types are added

1.3 High-Level Project Flow



■ Think of this pipeline like a factory: raw MRI images go in one end, and a trained AI doctor comes out the other end — ready to classify new scans instantly.

Chapter 2

Understanding the Data

2.1 Dataset Structure

The dataset is stored on Google Drive and is split into two main folders: **Training** and **Testing**. Inside each folder there are sub-folders — one per tumour class. The code walks through each sub-folder to collect image paths and their corresponding labels automatically.

```
■ MRI Images/ ■■■ Training/ ■ ■■■ glioma/ (e.g. 1321 images) ■ ■■■ meningioma/ (e.g. 1339 images) ■ ■■■ notumor/ (e.g. 1595 images) ■ ■■■ pituitary/ (e.g. 1457 images) ■■■ Testing/ ■■■ glioma/ ... (and so on)
```

2.2 What the Code Does When Loading Data

1	**os.listdir(train_dir)** Lists the 4 class-folder names: ['glioma', 'meningioma', 'notumor', 'pituitary']
2	**Loop through each folder** For every image file found, store the full path in train_paths and the folder name in train_labels
3	**shuffle(train_paths, train_labels)** Randomly mixes the data so the model doesn't learn in a biased order (e.g., all gliomas first)
4	**Same for test data** Exact same process repeated for the Testing folder

2.3 Why Shuffle the Data?

Imagine learning from a textbook that groups all glioma questions together, then all meningioma questions. You might accidentally think 'the next question is always the same type as the last one.' Shuffling prevents this bias — the model sees a random mix every epoch.

```
■ Key Line: train_paths, train_labels = shuffle(train_paths, train_labels) This keeps paths and labels in sync — index 5 in train_paths still matches index 5 in train_labels.
```

Chapter 3

Image Preprocessing & Augmentation

3.1 Why Do We Preprocess Images?

Neural networks are very picky — they need all inputs to be the same size and in the same numerical range. Raw MRI images come in different resolutions and formats. Preprocessing standardises them so the model can learn properly.

Step	What It Does	Why It Matters
Resize to 128×128	All images made the same size	Model needs fixed-size input
Divide by 255.0	Pixel values go from 0–255 → 0.0–1.0	Smaller numbers = faster, stable learning
Brightness tweak	Randomly brighten/darken (0.8x – 1.2x)	Mimics different MRI machine settings
Contrast tweak	Randomly adjust sharpness (0.8x – 1.2x)	Helps generalise to varied scans

3.2 The Three Helper Functions Explained Simply

augment_image(image)	**Makes One Image Ready for Training** — Takes a raw image array → randomly changes brightness/contrast → divides by 255 to normalise. It returns a float array between 0 and 1, which is exactly what the neural network expects.
open_images(paths)	**Loads a Batch of Images** — Loops through a list of file paths, calls load_img() to open each one at 128×128 size, runs augment_image() on it, and stacks them all into a single NumPy array. Think of it as a conveyor belt that processes images one by one.
encode_label(labels)	**Converts Words to Numbers** — The model can't understand the word 'glioma' — it only understands numbers. This function converts labels like ['glioma', 'notumor'] → [0, 2] (based on alphabetical order in the folder).

3.3 The Data Generator — Smart Batching

Loading all 5000+ images into RAM at once would crash your computer. The datagen() function is a **Python generator** — it loads only one small batch (e.g. 20 images) at a time, feeds them to the model, then loads the next batch. This is the standard professional approach for large datasets.

■ Analogy: Instead of carrying all books at once from the library (RAM overload), you carry them 20 at a time — read, return, fetch next 20. Same idea here.

Chapter 4

The Model — VGG16 Transfer Learning

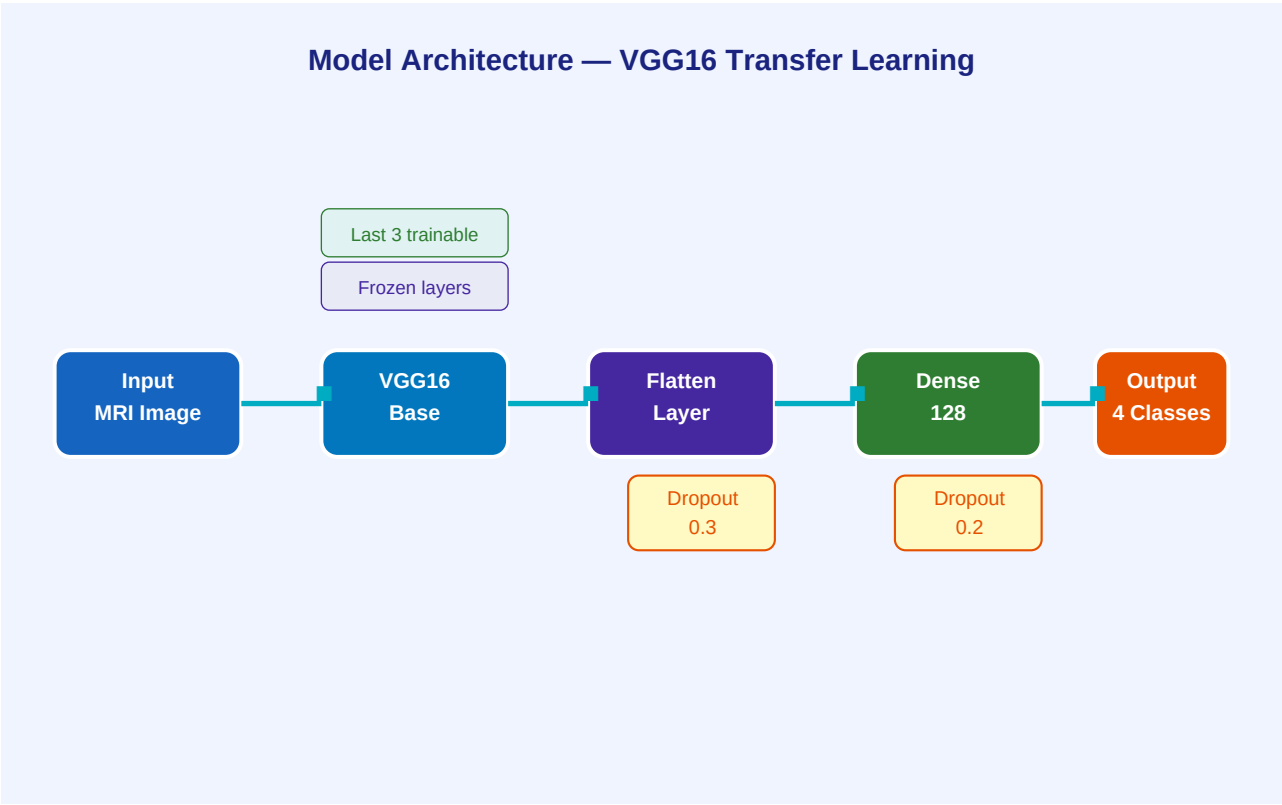
4.1 What is VGG16?

VGG16 is a deep neural network created by Oxford University in 2014. It was trained on **ImageNet** — a dataset of 1.4 million images across 1000 categories (cats, cars, flowers, etc.). It learned to recognise edges, textures, shapes, and complex patterns. We borrow this knowledge and apply it to MRI scans — that's **Transfer Learning**.

4.2 What is Transfer Learning? (The Simple Analogy)

■ Imagine a doctor who spent 10 years studying general medicine (ImageNet training). Now they want to specialise in brain tumours. Instead of starting from scratch, they use their existing knowledge of anatomy, biology, and imaging — and just learn the tumour-specific part. That's exactly what Transfer Learning does for AI.

4.3 Architecture Diagram



4.4 Layer-by-Layer Walkthrough

Layer	What It Does	In Plain English
VGG16 Base (frozen)	16 conv/pool layers — detect edges, textures, shapes	The eyes' borrowed from ImageNet training
Last 3 VGG16 layers (trainable)	Fine-tune high-level features for MRI	The doctor specialising in brain scans
Flatten()	Converts 3D feature map → 1D list of numbers	Unrolling a photo into a long list
Dropout(0.3)	Randomly turns off 30% of neurons during training	Prevents memorising — forces generalisation
Dense(128, relu)	128 neurons learn tumour-specific patterns	The decision-making layer
Dropout(0.2)	Randomly turns off 20% of neurons	Second safety net against overfitting
Dense(4, softmax)	Outputs 4 probabilities (one per class)	The final verdict: 'I'm 87% sure this is glioma'

Key Design Decisions Explained

include_top=False	We remove VGG16's original classification head (it was for 1000 ImageNet classes). We replace it with our own head for 4 tumour classes.
weights='imagenet'	Load pre-trained weights — 10 years of learning in one line of code.

layer.trainable= False	Freeze the base model so we don't accidentally destroy its learned knowledge during our training.
Last 3 layers trainable	Fine-tune the highest-level features. Early layers detect basic edges (same for any image), later layers detect complex shapes (useful to specialise).
Adam lr=0.0001	Adam is a smart optimiser. Very low learning rate = tiny, careful weight updates = stable training on pre-trained model.
sparse_categorical_crossentropy	Loss function for multi-class problems where labels are integers (0,1,2,3) not one-hot vectors.

Chapter 5

Training, Evaluation & Results

5.1 Training Parameters Explained

Parameter	Value	Meaning
batch_size	20	Feed 20 images at a time before updating weights
epochs	5	Go through the entire dataset 5 times
steps	len(train_paths) / batch_size	How many batches per epoch
learning_rate	0.0001	How big a step to take when adjusting weights

5.2 What Happens During Training?

1	**Forward Pass** Image goes in → model predicts a class → compares with true label
2	**Calculate Loss** How wrong was the prediction? Loss function measures this
3	**Backward Pass** Error signal travels backward through the network (backpropagation)
4	**Update Weights** Adam optimiser adjusts weights to reduce loss slightly
5	**Repeat** Same process for every batch in every epoch

5.3 Reading the Training Plot

After training, a graph shows Accuracy and Loss over epochs. A **healthy training** looks like: accuracy going UP, loss going DOWN. If accuracy stays flat or loss goes up, the model is struggling — maybe the learning rate is too high or data is too noisy.

5.4 Confusion Matrix — What It Tells You

The confusion matrix is a grid that shows: • Diagonal boxes = CORRECT predictions (we want these big) • Off-diagonal boxes = MISTAKES (model confused class A for class B) Example: If row=glioma and col=meningioma has a high number, the model often confuses glioma for meningioma — a clue to investigate more.

5.5 Classification Report — Key Metrics

Metric	Formula (Simple)	What It Means
Precision	True Positives / All Predicted Positives	Of all the times the model said 'glioma', how often was it right?
Recall	True Positives / All Actual Positives	Of all real glioma cases, how many did the model catch?
F1-Score	2 × (Precision × Recall) / (Precision + Recall)	Balanced score — useful when classes are imbalanced
ROC-AUC	Area under the ROC curve	Overall model quality — 1.0 = perfect, 0.5 = random guessing

Chapter 6

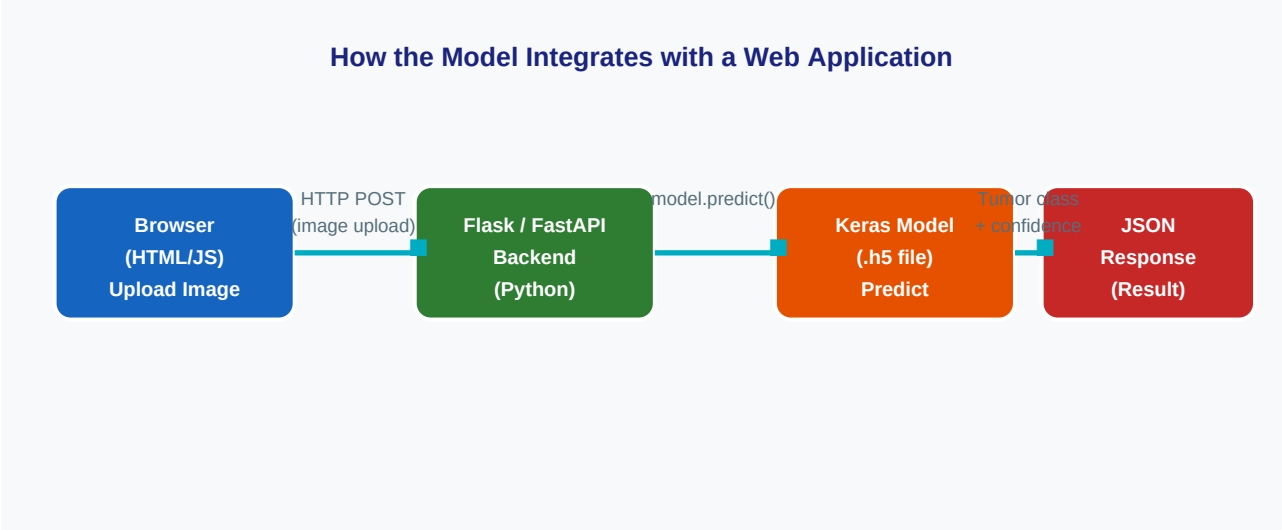
Web Integration, Demo Guide & Q&A;

6.1 Saving & Loading the Model

After training, the model is saved as **model.h5** — this is a binary file containing all the learned weights. You can load it anytime without retraining. This is what makes deployment possible.

```
model.save('model.h5') # Save after training model = load_model('model.h5') # Load in your web app
```

6.2 How Did I Integrate This Model With a Web Application?



Step-by-Step Web Integration:

1	**Save the trained model** Run model.save('model.h5') in Colab — download the file to your machine.
2	**Create a Flask/FastAPI app** pip install flask. Create app.py with a /predict route that accepts image uploads.
3	**Load model in backend** model = load_model('model.h5') — load once when the server starts, not on every request.

4	**Preprocess incoming image** Same steps as training: resize to 128×128, normalise to 0–1, add batch dimension.
5	**Predict and return JSON** predictions = model.predict(img_array). Return {'class': 'glioma', 'confidence': 0.92} as JSON.
6	**Frontend displays result** HTML + JS uploads the file via fetch() and shows the result on screen.

■ Key Insight for Audience: The model is just a file. Once trained, it's a tool — like a calculator. The web app is the interface. You separate the AI brain from the user interface. This is how real AI products are built.

6.3 Demo Script — How to Present on Webcam

■ Golden Rule: Never walk through code line by line. Show the STORY, not the syntax.

PART 1 — Demonstration (Show, Don't Tell)

- Open Google Colab — show it's already trained and the model is loaded.
- Show the dataset folders briefly — '4 types of tumours, training + testing folders'.
- Run the detect_and_display() function on 4 different test images (glioma, meningioma, notumor, pituitary).
- Show the confidence score — 'The model is 93% sure this is a pituitary tumour.'
- Show the confusion matrix plot — 'This is how we measure whether the model is trustworthy.'

PART 2 — Technical Explanation (Diagrams, Not Code)

- Pull up the Architecture Diagram from this PDF. Explain each block in 1 sentence.
- Say: 'VGG16 is a pre-trained model from Oxford. We borrowed its brain and added our own classifier on top.'
- Explain the pipeline: Load → Preprocess → Augment → Train → Evaluate.
- Show the ROC curve — 'Closer to top-left corner = better model. Ours is near perfect.'
- Show the web integration diagram — 'The model.h5 file is a plug-in. Any web app can use it.'

TIPS FOR WEBCAM PRESENCE

- Pause after each result — let the audience absorb it.
- Use the word 'confidence score' instead of 'softmax probability'.
- If asked about a line of code: 'Great question — that line does X, which is important because Y.'
- Keep a glass of water nearby. Breathe. Smile.

20 Interview & Demo Q&A;

Q1. Why did you use VGG16 instead of training a CNN from scratch?

Answer: Training a CNN from scratch needs millions of images and weeks of compute. VGG16 was already trained on 1.4 million images. We just fine-tune its top layers — saving time and getting better accuracy with limited medical data.

Q2. Why not use a more modern model like ResNet or EfficientNet?

Answer: VGG16 is simpler to understand and great for learning Transfer Learning concepts. In production, you'd experiment with ResNet50 or EfficientNetB0 which are faster. For this project, VGG16 gives excellent results and is easy to explain.

Q3. What is include_top=False and why do you use it?

Answer: The 'top' of VGG16 is its classification head — 3 fully connected layers for 1000 ImageNet classes. We don't need those. We remove them and add our own head for 4 tumour classes. include_top=False gives us just the feature extractor.

Q4. Why do you freeze most VGG16 layers?

Answer: Those frozen layers already know how to detect edges, textures, and shapes — knowledge from 1.4 million images. If we unfreeze them, our small dataset would overwrite that knowledge. We freeze to preserve it and only retrain the last few layers that learn high-level, tumour-specific features.

Q5. What is a Dropout layer and why do we have two of them?

Answer: Dropout randomly turns off a fraction of neurons during training. It prevents the model from memorising the training data (overfitting). Two dropouts with rates 0.3 and 0.2 act as two safety nets — the model is forced to learn robust patterns, not shortcuts.

Q6. Why is the learning rate set to 0.0001 (very small)?

Answer: We are fine-tuning a pre-trained model. A large learning rate would make big jumps and destroy the carefully learned weights from ImageNet. A tiny learning rate makes gentle, precise adjustments — like a surgeon's scalpel rather than a hammer.

Q7. What does sparse_categorical_crossentropy mean?

Answer: It's the loss function for multi-class classification. 'Categorical' = multiple classes. 'Sparse' = labels are integers (0, 1, 2, 3) rather than one-hot vectors ([1,0,0,0]). It measures how wrong the model's prediction is compared to the true label.

Q8. Why do you normalise images by dividing by 255?

Answer: Raw pixel values are 0–255 (integers). Neural networks train much better with small numbers close to zero. Dividing by 255 converts all pixels to 0.0–1.0. This also speeds up gradient descent and makes training more stable.

Q9. What is data augmentation and why do you do it?

Answer: Augmentation artificially creates variations of existing images — changing brightness and contrast here. This makes the model see 'different' versions of the same scan, so it learns general features rather than memorising specific images. It effectively multiplies your dataset.

Q10. Why do you use a data generator instead of loading all images at once?

Answer: Loading 5000+ images simultaneously would require gigabytes of RAM. A Python generator loads only one batch (20 images) at a time, feeds it to the model, discards it, then loads the next batch. This is memory-efficient and is standard practice in ML.

Q11. What does the Flatten layer do?

Answer: VGG16's last layer outputs a 3D feature map (4×4×512 for 128×128 input). The Dense layers expect a 1D input. Flatten unrolls the 3D array into a single vector of 8192 numbers. It's like unfolding a 3D cube into a straight line.

Q12. What does the Softmax activation in the output layer do?

Answer: Softmax converts raw scores (logits) into probabilities that sum to 1. For 4 classes, it might output [0.05, 0.87, 0.03, 0.05] — the model is 87% confident it's class 1 (glioma). We pick the class with the highest probability.

Q13. What is the ROC curve and what does AUC tell you?

Answer: ROC curve plots True Positive Rate vs False Positive Rate at different thresholds. AUC (Area Under Curve) summarises it — 1.0 means perfect classification, 0.5 means random guessing. High AUC across all 4 classes means our model is reliable.

Q14. What is the confusion matrix and what patterns should you look for?

Answer: It's a grid where rows=actual labels, columns=predicted labels. The diagonal shows correct predictions. Off-diagonal cells show confusions. For medical AI, we especially worry about false negatives (real tumour classified as no tumour) — those are dangerous.

Q15. Why is Recall more important than Precision in medical diagnosis?

Answer: Precision = how many 'positive' predictions were correct. Recall = how many actual positives were found. In medicine, missing a real tumour (low recall) is far more dangerous than a false alarm (low precision). We'd rather over-diagnose than miss cancer.

Q16. How did you integrate this model with a web application?

Answer: Save the trained model as model.h5. Create a Flask backend with a /predict API endpoint. The frontend uploads an MRI image → Flask receives it → preprocesses exactly as in training → model.predict() returns probabilities → Flask sends JSON response → webpage displays the result.

Q17. Why use .h5 format to save the model?

Answer: HDF5 (.h5) is a hierarchical data format that stores both the model architecture and trained weights in one file. You can load it on any machine with Keras and make predictions instantly — no retraining needed. It's the standard Keras save format.

Q18. What are the limitations of this model?

Answer: Trained on a specific dataset — may not generalise to all MRI machines or populations. 5 epochs is relatively short training. No explainability (can't highlight which part of the scan triggered the prediction). Not clinically validated — should never replace a doctor.

Q19. If accuracy is high but the model is still useless, why?

Answer: Class imbalance problem. If 90% of data is 'no tumor', a model that always predicts 'no tumor' gets 90% accuracy but misses every real tumour. This is why we use F1-score, precision, recall, and ROC-AUC — they reveal this kind of deceptive accuracy.

Q20. What would you improve if you had more time?

Answer: More epochs (10–20). Try ResNet50 or EfficientNet. Add Grad-CAM (heatmap showing which part of MRI the model looked at). Use cross-validation. Deploy with a proper web interface. Add more augmentation (rotation, flipping). Use a larger, more diverse dataset.